

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are increasingly integrated with graph-structured data across a wide range of relational learning problems, from node-level classification to broader graph reasoning tasks that have traditionally been the province of Graph Neural Networks (GNNs). While this integration leverages rich relational information to improve task performance across many domains, their robustness against adversarial attacks remains largely unexplored. We take the first step toward understanding the vulnerabilities of the broad family of graph-aware LLMs by leveraging established adversarial attack methods developed for graph-based models, spanning both poisoning (training-time attacks) and evasion (test-time attacks). We ground this investigation in two representative models that typify the frozen-LLM-with-learned-projector paradigm, LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024). Beyond adapting existing attacks, we uncover a new attack surface for LLAGA in which an adversary injects malicious nodes as placeholders into the node sequence template to severely degrade its performance. Our systematic analysis shows that design choices in how graphs are encoded can broadly shape a model’s exposure to attack, yielding transferable findings that: (1) the node sequence template in LLAGA amplifies its vulnerability; (2) the GNN encoder used in GRAPH-PROMPTER confers greater robustness; and (3) both encoding paradigms remain susceptible to imperceptible feature perturbation attacks. Finally, we propose an end-to-end defense framework, GALGUARD, that combines an LLM-based feature correction module to mitigate feature-level perturbations with adapted GNN defenses that protect against structural attacks, offering a general template for hardening graph-aware LLMs¹

1 Introduction

Graphs, characterized by nodes and edges that represent entities and their relationships, are a general-purpose abstraction for modeling complex structures across a broad spectrum of real-world domains, including social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021), among many others. Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018), designed specifically for graph-structured data, excel across a wide

range of tasks such as node classification and link prediction. More recently, the success of large language models (LLMs) such as GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023), trained on vast amounts of textual data, has sparked broad interest in their potential to enhance the full range of graph-related tasks. This fusion of graph data with LLMs (called *graph-aware LLMs* (Xie et al. 2023)) has opened a rapidly expanding line of research, in which LLMs are increasingly being adopted to perform tasks that were once the exclusive domain of GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Indeed, a growing body of work shows that graph-aware LLMs can match or outperform GNNs across a variety of graph-related tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), positioning them as general and powerful alternatives.

Despite their growing popularity and expanding range of deployment, the vulnerabilities of graph-aware LLMs to attacks remain largely unexplored. A rich body of adversarial attacks on GNNs, spanning both poisoning (training-time attacks) and evasion (test-time attacks) (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), typically operates under a black-box setting in which the attacker has no knowledge of the model’s internal workings. These attacks undermine *model integrity* by intentionally degrading performance through subtle alterations to graph structure or node features. While GNN research has predominantly emphasized structural attacks, the inherently textual nature of graph-aware LLMs makes imperceptible perturbations to node features a broadly relevant concern that has so far been largely overlooked.

Motivated by three broad observations: the ongoing shift from GNNs to LLMs across graph tasks, the black-box nature of current adversarial attacks on GNNs, and the potentially new vulnerabilities introduced by feature perturbations, we ask the critical and widely applicable question of whether *adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm as a whole*. Answering this question requires attention to how graphs are integrated into LLMs, since the success of graph-aware LLMs across tasks *depends heavily* on this integration. While processing node features is comparatively straightforward given

¹Our code will be released upon acceptance.

the proficiency of LLMs with textual data, incorporating graph structure is more involved and has motivated a diverse set of techniques for capturing relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). To answer our research question in a way that generalizes across this design space, we first organize existing graph-aware methods into two broad classes according to how they encode graphs into LLMs: one that represents graph data as *textual descriptions*, converting graph information into natural language, and another that employs *learned projectors* to bridge the inherent differences between the graph and text domains. Because conventional text-based adversarial attacks on LLMs already suffice to compromise models that consume textual descriptions of graphs, we concentrate our investigation on the learned projector family, using two computationally efficient and widely representative models, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024), both of which build on a frozen pre-trained LLM.

Our analysis reveals that design choices in how graphs are encoded for LLMs can unexpectedly enlarge the attack surface, a lesson that extends beyond any single model. Notably, we identify a new attack surface in LLAGA through which an attacker can severely degrade performance by injecting malicious nodes as placeholders within the node sequence template. In addition, we propose an end-to-end graph-aware LLM defense, GALGUARD, which integrates an LLM-based feature corrector with adapted GNN-based structural defenses to provide robust, broadly applicable protection. To our knowledge, we are the first to systematically investigate the vulnerability of graph-aware LLMs to adversarial attacks. The contributions of the paper are summarized as follows:

- We leverage representative adversarial attacks for GNNs, specifically NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to study how broadly poisoning and evasion attacks transfer to the graph-aware LLM paradigm.
- We introduce a general taxonomy of graph-aware LLMs organized by their encoding approaches, clarifying how different families of methods integrate graph structure into LLMs.
- We discover a new attack surface in LLAGA, showing that injecting malicious nodes as placeholders into the node sequence template can degrade its performance, and we discuss why such template-based designs are broadly at risk.
- We provide a holistic assessment of graph-aware LLM vulnerabilities by performing imperceptible attacks on textual node features, a threat vector that applies wherever node attributes are textual yet has been largely overlooked in GNN research.
- We propose GALGUARD, a novel end-to-end and plug-and-play defense strategy that integrates LLM-based feature correction with the adaptation of existing GNN-based structural defenses.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

The many ways of encoding graphs for LLMs can be organized into two broad and largely exhaustive families: *textual descriptions of graphs* (where graph information is encoded as natural language) and *learned graph projectors* (where a projector encodes graph data into embeddings or structured forms that LLMs can directly process). We briefly describe the representative encoding approaches that fall within each family. Figure 5 illustrates both families. A full description appears in Appendix A.1, and related work on GNN attacks in Appendix A.2.

Graph Representation as a Textual Description. This broad family converts graph data into natural language so that LLMs can process it as textual descriptions, often with task-specific instructions used to query the model (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Peruzzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). The family spans a wide range of settings: in multimodal applications, Liu et al. (2023) and Su et al. (2022) linked molecule graphs with textual descriptions. Furthermore, Huang et al. (2023a) investigated incorporating graph structure into LLM prompts. Their findings reveal that LLMs tend to process graph-related prompts as contextual paragraphs, underscoring the general importance of prompt design, as reflected in studies such as (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). A parallel body of work uses LLMs to enhance node features and graph structures, to act as predictors, or to serve as feature extractors for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that combines LLMs and GNNs, and Duan et al. (2023) introduced an effective approach to enhance textual graph learning. While these methods offer a straightforward and accessible way to apply LLMs to graph tasks, they often struggle to fully capture structural intricacies (especially for large graphs) and depend heavily on prompt engineering.

Graph Representation as a Learned Projector. To overcome the limitations of textual representations, the learned projector family transforms graph data into embeddings or graph vectors that LLMs can process more directly, and it encompasses a wide variety of architectures. Representative instances include LLAGA, which uses node-level templates and a linear projector (Chen et al. 2024), and methods that incorporate GNN layers into a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Still others use graph neural prompts to encode knowledge graphs (Tian et al. 2024), develop prompt-based node feature extractors with a GNN adapter (Huang et al. 2023b), or condense graph information into fixed-length prefixes using graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024).

This Work: We focus on projector methods that build on *frozen pre-trained LLMs*, a design choice shared across much of this family because it is computationally efficient and avoids the substantial costs of retraining (Chen et al. 2024;

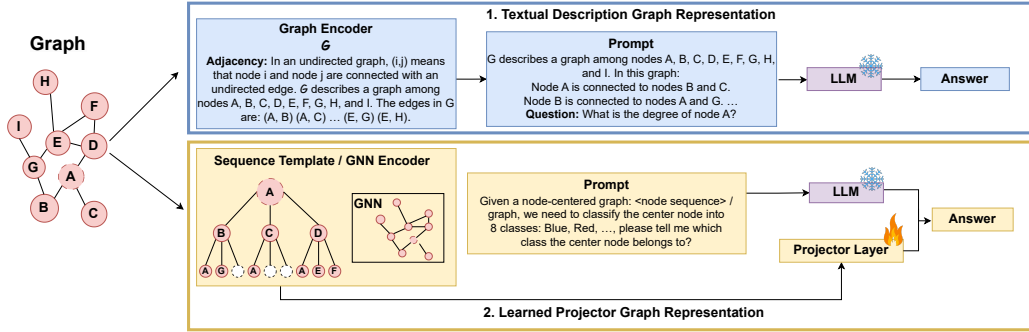


Figure 1: **Graph encoding-based adaptations of LLMs.** In the textual description graph representation, graphs are explained textually (top part). In the learned projector representation, graphs are encoded either by node templates that turns the graph into sequences or by GNNs (bottom part). Note that in both cases, the LLM is frozen. Only the projector that maps the graph into tokens is trained.

Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). As representatives spanning the two dominant structural encodings within this family, we study LLAGA (Chen et al. 2024), which employs node-level templates with a linear projector, and GRAPH PROMPTER (Liu et al. 2024), which uses a GNN with a linear projector. We do not separately study textual description methods, since they tend to struggle with scalability and do not introduce the novel, graph-specific vulnerabilities that are the focus of our investigation; conclusions about them follow largely from prior work on text-based attacks on LLMs.

3 Threat Model

We build our threat model on established research on adversarial attacks in graphs and extend its applicability to the broad class of graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). The model captures a range of realistic deployment settings. In social networks, for instance, attackers often control only a handful of bot accounts; this limited access aligns naturally with our threat model, in which adversaries manipulate a small subset of nodes to evade detection or influence legitimate user classification. As another example, on Wikipedia, hoax articles with sparse connections to legitimate ones can be strategically linked to manipulate the classification of genuine content, causing misclassification of real articles’ categories. Such transferability is especially concerning because these models are increasingly poised for deployment in high-stakes applications such as recommendation systems, social media analysis, and even healthcare, where they may progressively replace GNNs. Following the broader literature on adversarial attacks in graphs, we characterize the threat model along three dimensions—model access, adversarial capabilities, and attack strategies—before defining the attacker’s goal.

Model Access. The attacker operates in a general black-box setting, with no knowledge of the model’s architecture or parameters. The attacker can influence the model only indirectly, by interacting with the training or test data—an

assumption that holds across most practical deployments.

Adversarial Capabilities. Consistent with the standard poisoning (evasion) setting, the attacker has access to the training (inference) data of the target model. Within a predefined budget and under unnoticeability constraints (intended to avoid detection), the attacker may modify or inject data.

Attack Strategies. The attacker draws on two broad and complementary strategies: feature manipulation (altering node features) and structure manipulation (adding or removing edges), which together cover the principal input modalities of graph-aware models.

Attacker’s Goal. The attacker aims to compromise model integrity by degrading overall performance, specifically by increasing the misclassification rate of the classification model.

4 Experimental Setup

Dataset. We evaluate on three widely used text-attributed graph benchmarks—Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016)—that together span a range of scales and sparsities from small to medium and serve as standard testbeds for the class of models we study (Chen et al. 2023a, 2024; Liu et al. 2024). To probe how our findings generalize to large, real-world graphs, we additionally evaluate on the substantially larger ArXiv dataset (Hu et al. 2020) in a dedicated experiment (see Appendix A.5). We adopt the standard train/validation/test splits of 6:2:2 for the three smaller datasets and 5:2:3 for ArXiv, following (Chen et al. 2024). Detailed statistics are provided in Table 1.

Table 1: Dataset statistics.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. We adopt *node classification* as our primary task. It is among the most widely used settings for evaluating graph machine learning models and is the canonical setting for

adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023), making it a natural and representative choice for studying vulnerabilities that we expect to extend to related graph tasks.

Evaluation Metrics. We report *accuracy*, which measures the proportion of correctly predicted labels, as our evaluation metric. Each experiment is repeated three times, and we report the mean and standard deviation. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We now motivate our selection of graph-aware LLM methods and adversarial attack methods, chosen to be representative of the broader design space so that our findings speak to the paradigm rather than to isolated models.

Graph-Aware LLM Methods. We study two representative graph-aware LLM methods: LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024). Both build on a *frozen pre-trained LLM*, a shared design that makes them computationally efficient and readily adaptable across graph tasks and LLM backbones. LLAGA uses node-level templates with a linear projector, while GRAPH PROMPTER employs a GNN-based encoder with a linear projector; between them they cover the two dominant strategies for encoding structure in this family. In both cases, the projector is a 2-layer multi-layer perceptron (MLP). The foundational LLMs for LLAGA and GRAPH PROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Adversarial Attack Methods. We conduct our investigation using the poisoning and evasion variants of two representative and widely adopted adversarial attack methods, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), which between them span targeted and untargeted objectives. NETTACK is a targeted attack that finds minimal perturbations to misclassify specific nodes, while METAATTACK performs untargeted attacks by treating the graph structure as a hyperparameter to optimize. For NETTACK, we follow the protocol of (Li et al. 2023; Zügner and Günnemann 2019), sequentially targeting each node and aggregating the results, which effectively yields an untargeted evaluation. Across all experiments, we perturb only 10% of the edges to respect the unnoticeability constraint, as in (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Full details appear in Appendix A.2.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We analyze poisoning and evasion attacks by adapting the NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) methods to the graph-aware LLM setting. For poisoning attacks on LLAGA, we use a perturbed graph to generate the node-level sequences and then retrain the projector. For GRAPH PROMPTER, we use the perturbed graph to re-encode the graph with a GAT and retrain its projector. For evasion attacks, both models are trained on unperturbed data, and perturbations are introduced

only during inference. These procedures follow directly from each model’s encoding pipeline and thus extend naturally to other members of the frozen-LLM projector family.

Results. Our experiments surface several insights that recur across models, datasets, and attack methods, as summarized in Table 2. First, poisoning attacks have a limited impact on graph-aware LLMs, with a maximum performance degradation of 11% across both attack methods. This points to a broadly protective effect of the models’ robust representations and their reliance on additional context from the LLM. In contrast, evasion attacks prove highly effective, causing substantial degradation of up to 51%. Second, comparing attack methods, METAATTACK generally outperforms NETTACK for both poisoning and evasion across all datasets, which we attribute to METAATTACK’s ability to craft small but highly effective perturbations by treating graph edges as hyperparameters. Finally, a consistent trend emerges with respect to graph density: denser datasets are more vulnerable to attack than sparser ones. For instance, LLAGA exhibits a 51% accuracy drop on the denser Cora under an evasion attack, but only a 19% drop on the sparser PubMed dataset—a dependence on structural density that we expect to hold more broadly.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

Our experiments (Table 2) show that LLAGA is more susceptible to attacks than GRAPH PROMPTER, and tracing why reveals a general principle rooted in *sensitivity to structural perturbations*. LLAGA integrates graph structure in two ways: by generating a fixed-length node-level sequence for each node and by computing a graph Laplacian embedding. Each position in this sequence uniquely corresponds to a relative structural position within the original graph. Because the node embeddings are derived directly from the perturbed graph structure, such attacks strongly influence the information passed to the LLM, leading to performance degradation. In contrast, GRAPH PROMPTER’s textual node embeddings remain unchanged under structural perturbations, since only the GNN component is affected. This design lets unperturbed textual features augment the perturbed structure, conferring robustness. More generally, the comparison suggests that encodings which tightly bind representations to raw structure are inherently more exposed than those that retain an independent, unperturbed feature channel.

6 Node Sequence Template Injection Attacks

We demonstrate a new attack surface in LLAGA that arises from exploiting the neighborhood detail template used to construct the node sequences—a vulnerability class relevant to any method built on fixed-shape neighborhood templates. The attack is illustrated in Figure 2 and the algorithm is given in Appendix A.4. To obtain the node sequence for a particular node u , the neighborhood detail template in LLAGA first constructs a *fixed-shape* computational tree for each node, where the children of a node are its sampled 1-hop neighbors. This process repeats to sample neighbors for each subsequent

Table 2: Performance of NETTACK and METAATTACK across the Cora, Citeseer, and PubMed benchmarks for the graph-aware LLMs LLAGA and GRAPH PROMPTER, illustrating attack behavior across datasets of differing scale and density.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

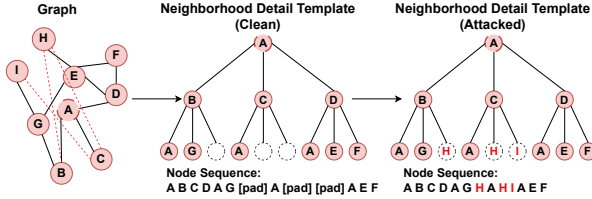


Figure 2: Illustration of the node sequence template injection attacks, a general family of placeholder-exploiting strategies. The instance shown here is the Non-adjacent Injection attack (NI), where red dashed lines denote the attacker’s edge perturbations. For the Supernode Injection (SI) and Multiple Supernode Injection (MSI) attacks, the supernode(s) are injected in place of the placeholders. Here, $k = 3$.

level of the tree. When a node has too few neighbors, a placeholder fills the remaining slots to maintain the fixed size of, e.g., $k = 10$ children, consistent with LLAGA’s original design (see “Clean” in Figure 2 and lines 1–14 in the algorithm). It is precisely this fixed structure that opens up a new attack surface.

Exploiting this neighborhood detail template algorithm, an attacker can inject malicious nodes into placeholder positions through graph manipulation to significantly degrade model performance. We instantiate this idea with three complementary strategies that together illustrate the breadth of the attack. The first, *Non-Adjacent Injection (NI)*, injects nodes that lie outside the 2-hop neighborhood of a target node u (see “Attacked” in Figure 2 and lines 21–27). The second, *Supernode Injection (SI)*, injects a high-degree node as a placeholder, again ensuring it lies outside the target node’s 2-hop neighborhood. The third, *Multiple Supernode Injection (MSI)*, is a variant of *SI* that injects multiple distinct supernodes as placeholders (lines 35–43). The common rationale is that including uninformative nodes in the node sequence corrupts the target node’s representation, yielding a poorly trained projection layer and thereby compromising the integrity of both learning and inference.

Results. As shown in Table 3, LLAGA is highly resilient to poisoning across the Cora and Citeseer datasets, yet suffers a substantial drop under evasion, with accuracies decreasing to 0.42, 0.36, and 0.30 on Cora and to 0.36, 0.30, and 0.24 on Citeseer for the *NI*, *SI*, and *MSI* attacks, respectively.

Table 3: Results of the node sequence template injection attack across datasets and injection strategies. *NI*, *SI*, and *MSI* denote the non-adjacent, single supernode, and multiple supernode injection attacks, respectively. Clean refers to model performance without any attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 $\pm$ 0.07	—
	Poisoning	0.86 $\pm$ 0.03	0.84 $\pm$ 0.06	0.82 $\pm$ 0.04
	Evasion	0.42 $\pm$ 0.08	0.36 $\pm$ 0.05	0.30 $\pm$ 0.05
Citeseer	Clean	—	0.71 $\pm$ 0.04	—
	Poisoning	0.69 $\pm$ 0.05	0.66 $\pm$ 0.03	0.62 $\pm$ 0.02
	Evasion	0.36 $\pm$ 0.06	0.30 $\pm$ 0.04	0.24 $\pm$ 0.05
PubMed	Clean	—	0.90 $\pm$ 0.04	—
	Poisoning	0.88 $\pm$ 0.04	0.85 $\pm$ 0.05	0.81 $\pm$ 0.03
	Evasion	0.85 $\pm$ 0.07	0.82 $\pm$ 0.04	0.76 $\pm$ 0.03

On the larger PubMed dataset, by contrast, the model is markedly more resilient to all injection attacks, with only a modest evasion-time drop (0.85, 0.82, and 0.76). This scale-dependent pattern has a general explanation: in smaller, more tightly connected graphs such as Cora and Citeseer, a localized injection has a concentrated impact that severely disrupts the model’s ability to reason about the local neighborhood, whereas in the much larger PubMed graph the same attack is diluted, producing less degradation.

Relative to traditional graph adversarial methods, node injection attacks are consistently more effective. For instance, the *MSI* method attains evasion accuracies of 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, whereas METAATTACK only reaches 0.44, 0.50, and 0.73 (Table 2)—underscoring the broad potency of template-targeted attacks.

7 Poisoning and Evasion Attack via Feature Perturbation

Existing graph adversarial attacks focus predominantly on structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019), with the exception of NETTACK (Zügner, Akbarnejad, and Günnemann 2018), whose feature attack does not transfer to the textual domain. Because structural attacks are only moderately successful against robust models such as GRAPH-

PROMPTER, and because textual features have proven central to model robustness, we investigate a complementary and broadly applicable threat: unnoticeable feature perturbation attacks, which apply to any model that consumes textual node attributes.

We build on the imperceptible attack strategies of (Boucher et al. 2022) with one key modification: we use a task-agnostic translation objective (English to French) to generate perturbations. The goal is to alter the input text so that it remains visually identical yet yields an incorrect translation, thereby corrupting the encoded semantics; the intuition is that if the translation of a text is wrong, its encoded semantics are wrong as well. Using a task-agnostic objective keeps the attack independent of the downstream task, broadening its applicability. We employ two imperceptible attacks: *Homoglyphs*, which replaces characters with visually similar ones, and *Reorderings*, which uses direction control characters to alter the rendering sequence. Examples of Homoglyphs and Reorderings appear in Appendix A.6.

Following (Boucher et al. 2022), we optimize these perturbations with differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between the model’s output on the perturbed and unperturbed text. The perturbation budget is set to 10% of the average length of each textual feature.

Results. As shown in Table 4, Reordering attacks are consistently more potent than Homoglyph attacks across all models and datasets. On Cora and CiteSeer, LLAGA is vulnerable to both attack types, with poisoning causing accuracy drops of 27% and 23% under Homoglyph and a larger 35% and 25% under Reordering, respectively; GRAPHPROMPTER proves more resilient in both cases. PubMed exhibits the same qualitative trends, though the overall impact is less pronounced than on the smaller datasets—indicating that these vulnerabilities hold across dataset scales while their magnitude varies.

Comparing against structural perturbations reveals a fundamental and broadly relevant shift in the locus of vulnerability, as shown in Figure 3. Feature perturbation attacks (specifically Reordering) outperform structural attacks (METAATTACK) by a wide margin on both LLAGA and GRAPHPROMPTER. This indicates that graph-aware LLMs generally weight input features more heavily than graph structure, leaving them more exposed to feature perturbations. We probe this further by combining both attack types in a unified setting in the next section.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We propose a unified attack that combines feature and structural perturbations into a single, more imperceptible and more effective adversarial strategy. We hypothesize that jointly applying both attack types degrades model accuracy more than either alone, while keeping alterations to both graph structure and features imperceptible. Because it targets the two input modalities common to graph-aware models, this strategy is broadly applicable across the paradigm.

For the structural component, we use METAATTACK,

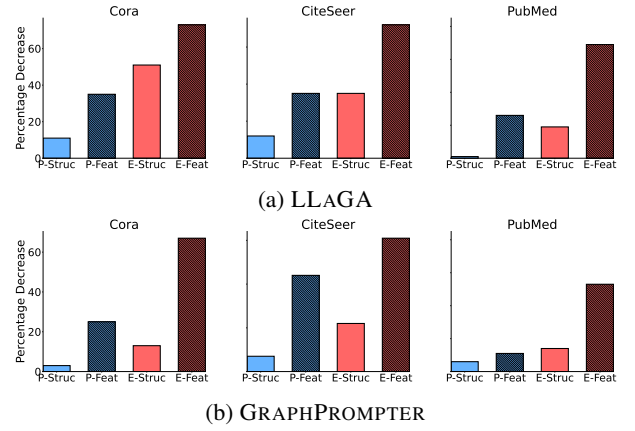


Figure 3: Comparison of structure- and feature-based adversarial attacks on graph-aware LLMs. Attack success is measured by the percentage decrease (y-axis) from the original accuracy. P-Struc, P-Feat, E-Struc, and E-Feat denote the poisoning and evasion attacks based on structural and feature perturbation. The consistent gap across both models highlights a paradigm-wide shift toward feature-level vulnerability.

which modifies graph structure by subtly adjusting edges. For the feature component, we combine the Homoglyph and Reordering attacks described in the previous section. Together, these perturbations form a highly imperceptible attack that simultaneously targets the relational and textual information on which graph-aware LLMs generally rely.

Results. As shown in Table 5, the unified attack, which jointly applies structural and feature perturbations, is markedly more effective than either perturbation method alone at degrading performance, and this advantage holds consistently across datasets. The result offers a broadly applicable insight into the architecture of graph-aware LLMs: unlike traditional GNNs, which are chiefly affected by structural perturbations, these models are substantially vulnerable to feature-level attacks. The effectiveness of the unified attack indicates that graph-aware LLMs rely on the consistency of both graph structure and node features, leaving them highly susceptible to such combined, inconspicuous modifications—a susceptibility we expect to generalize to other models sharing this reliance.

9 Ablation Study

9.1 Varying Feature Perturbation

To better understand the relationship between perturbation level and attack success, we study the effect of varying the fraction of perturbed features. Our analysis of the Reordering attack, whose behavior mirrors that of the Homoglyph attack, reveals a consistent trend as the perturbation percentage ranges from 1% to 10%. Across all models and datasets (Cora, Citeseer, and PubMed), reducing the fraction of perturbed features yields a less successful attack, as expected, and the uniformity of this trend across settings suggests it is a general property rather than a dataset-specific artifact. No

Table 4: Performance under feature perturbation attacks across datasets and models, with percentage drop from clean accuracy in parentheses.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 $\pm$ 0.05 (27%)	0.52 $\pm$ 0.04 (13%)	0.41 $\pm$ 0.04 (54%)	0.38 $\pm$ 0.03 (37%)
	Reordering	0.58 $\pm$ 0.06 (35%)	0.45 $\pm$ 0.05 (25%)	0.24 $\pm$ 0.05 (73%)	0.20 $\pm$ 0.03 (67%)
Citeseer	Homoglyph	0.54 $\pm$ 0.04 (23%)	0.65 $\pm$ 0.04 (7%)	0.32 $\pm$ 0.03 (55%)	0.52 $\pm$ 0.05 (26%)
	Reordering	0.46 $\pm$ 0.03 (35%)	0.39 $\pm$ 0.02 (44%)	0.20 $\pm$ 0.05 (72%)	0.27 $\pm$ 0.02 (61%)
PubMed	Homoglyph	0.72 $\pm$ 0.04 (20%)	0.83 $\pm$ 0.04 (8%)	0.36 $\pm$ 0.06 (60%)	0.55 $\pm$ 0.03 (39%)
	Reordering	0.67 $\pm$ 0.05 (26%)	0.80 $\pm$ 0.03 (11%)	0.28 $\pm$ 0.04 (69%)	0.42 $\pm$ 0.04 (53%)

tably, as the perturbation level decreases, the success rates of poisoning and evasion attacks tend to converge. This occurs because, with fewer perturbations, the adversarial signal becomes too weak to meaningfully affect the model, regardless of whether it was introduced during training (poisoning) or inference (evasion). A detailed plot illustrating these trends appears in the Appendix (Figure 2).

9.2 Does the Choice of LLM Affect Attack Performance?

We assess whether the choice of foundation LLM influences the adversarial robustness of graph-aware LLMs, an important question for gauging how far our conclusions carry across backbones. LLAGA uses Vicuna-7B (Chiang et al. 2023) and GRAPHPROMPTER uses Llama2-7B (Touvron et al. 2023). We replace LLAGA’s backbone with Llama2-7B and compare performance under poisoning and evasion attacks across all datasets. Both configurations achieve comparable clean accuracy (e.g., Vicuna: 0.89 ± 0.07 ; Llama2: 0.90 ± 0.03 on Cora) and exhibit similar vulnerability under attack, and the same trend holds when swapping LLMs in GRAPHPROMPTER. These results indicate that the choice of foundation LLM has minimal impact on adversarial robustness, suggesting that our findings are largely backbone-agnostic. The full results appear in Table 2 of the Appendix.

10 Defense: GALGUARD

Existing GNN defenses against adversarial attacks are broadly ineffective for graph-aware LLMs because of these models’ dual vulnerability to structural and feature perturbations. Many defenses rely on node feature similarity to prune or reweight edges (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), but all assume access to clean features. When features are perturbed, these methods fail (see Figure 3 in the Appendix). Likewise, defenses that decouple structure and features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021), and even OCR-based sanitization (Boucher et al. 2022), break down under coordinated attacks. This dual vulnerability, which we argue is inherent to the paradigm rather than to any single model, calls for a new, integrated defense.

To meet this need, we propose graph-aware LLM defense (GALGUARD), an end-to-end and broadly applicable defense strategy that combines two complementary components: (1)

Table 5: Model accuracy under the unified imperceptible attack across datasets and models. Percentage drop from clean accuracy in parentheses.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 $\pm$ 0.05 (61%)	0.28 $\pm$ 0.08 (53%)
	Evasion	0.14 $\pm$ 0.06 (84%)	0.11 $\pm$ 0.05 (82%)
Citeseer	Poisoning	0.26 $\pm$ 0.04 (63%)	0.22 $\pm$ 0.03 (69%)
	Evasion	0.10 $\pm$ 0.09 (86%)	0.13 $\pm$ 0.07 (81%)
PubMed	Poisoning	0.39 $\pm$ 0.07 (57%)	0.32 $\pm$ 0.04 (64%)
	Evasion	0.12 $\pm$ 0.05 (87%)	0.19 $\pm$ 0.04 (79%)

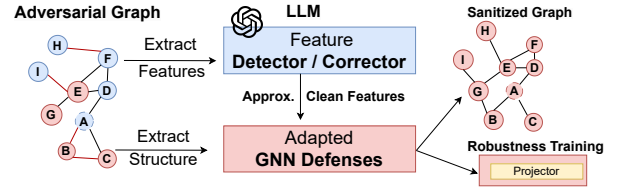


Figure 4: **Overview of GALGUARD defense.** Perturbed node features and edges are colored in blue and red, respectively. The LLM identifies perturbations in node features extracted from an adversarial graph and generates a corrected version. The refined features, together with the perturbed edges, are then passed through existing GNN defenses, which can either yield a sanitized graph or be adapted for projectors’ robustness training. The modular design makes the framework applicable across the range of frozen-LLM projector models.

an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As shown in Figure 4, an LLM serves as an adaptive mechanism that analyzes textual node features for anomalous patterns. It provides approximate corrections that restore feature integrity, thereby mitigating feature-level perturbations. Because it operates directly on textual features, this LLM-based correction offers a practical and model-agnostic solution to feature vulnerabilities. The specific instructions and prompts are given in Appendix A.6. For our experiments, we employ GPT-4 Turbo as the LLM corrector, though the design is agnostic to the particular corrector model used.

Graph Structure Robustness Strategy. To enhance structural robustness, we employ a two-step approach that adapts naturally to either encoding strategy. First, we adopt a *graph purification heuristic* (Wu et al. 2019) as a preprocessing step: for each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and its neighbors’ embeddings $\phi(x_u)$ and remove edges with low similarity. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020). We apply GNNGuard’s principles directly to the message-passing layers of GRAPHPROMPTER. For LLAGA, we propose a novel adaptation that incorporates the purification step into the construction of the computational tree for its node sequence.

We further introduce a learnable *global structural context embedding* (M_{global}) as a parameter in LLAGA’s projector. By concatenating M_{global} with the purified node sequence embedding and training them jointly, the model learns to operate robustly on potentially perturbed graph inputs. The plug-and-play design of GALGUARD means other defenses, e.g., Pro-GNN (Jin et al. 2020), can be substituted for GNN-Guard, making the framework readily extensible to defenses beyond those we evaluate.

10.1 Results

Table 6 summarizes accuracy under METAATTACK alongside our defenses: GALGUARD_p, which combines a feature corrector with a graph purification module, and the full GALGUARD, which integrates the feature corrector, the graph purification module, and GNNGuard (or its adaptation). Across all evaluated datasets (Cora, CiteSeer, and PubMed) and for both poisoning and evasion attacks, our defenses consistently mitigate the impact of METAATTACK, and this consistency across settings speaks to the general applicability of the approach. In particular, GALGUARD_p is an effective foundational step, showing that addressing feature perturbations with the LLM corrector and applying initial graph purification together substantially improve robustness. The full GALGUARD consistently yields larger robustness gains than GALGUARD_p, underscoring the synergistic benefit of integrating the GNN-inspired structural defense.

Table 6: Performance of METAATTACK and the proposed defense GALGUARD under poisoning and evasion attacks across datasets and models. GALGUARD_p combines feature correction with graph purification.

Data Method		LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79±0.03	0.44±0.06	0.58±0.05	0.52±0.04
	GALGUARD _p	0.82±0.05	0.62±0.04	0.59±0.03	0.56±0.04
	GALGUARD	0.85±0.04	0.83±0.04	0.60±0.05	0.59±0.03
CiteSeer	MetaAttack	0.63±0.04	0.50±0.03	0.65±0.06	0.56±0.03
	GALGUARD _p	0.64±0.03	0.56±0.02	0.67±0.05	0.60±0.06
	GALGUARD	0.67±0.04	0.62±0.04	0.68±0.04	0.64±0.05
PubMed	MetaAttack	0.89±0.08	0.73±0.04	0.85±0.02	0.77±0.03
	GALGUARD _p	0.90±0.06	0.78±0.03	0.85±0.04	0.81±0.04
	GALGUARD	0.90±0.01	0.87±0.02	0.89±0.02	0.89±0.01

11 Conclusion

In this paper, we take a substantial first step toward uncovering the vulnerabilities of graph-aware LLMs to adversarial attacks, a largely unexplored area with broad significance. Our work shows that the mechanisms used to integrate graph data are a critical and general source of vulnerability. We discovered new attack surfaces, demonstrating that design choices such as the node sequence template in LLAGA substantially increase a model’s susceptibility—a lesson that extends to other template-based encodings. Our findings further show that feature perturbation attacks are highly effective

(in contrast to traditional GNNs), frequently outperforming structural attacks, and that a unified attack combining both perturbations can render graph-aware LLMs near-useless.

To address this, we propose an end-to-end defense, GALGUARD, that integrates an LLM-based feature corrector with adapted GNN-based structural defenses to provide robust, broadly applicable protection. As these models grow more prevalent across applications spanning social networks, healthcare, finance, and beyond, ensuring their resilience to adversarial exploitation becomes ever more essential. We hope this work serves as a foundation for further research, inspiring deeper exploration of the vulnerabilities and defenses of graph-aware LLMs across models, tasks, and domains, and fostering both theoretical insight and practical safeguards for their real-world deployment.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.

- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.

- Olatunji, I. E.; Nejd, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.

- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix is organized as follows. Appendix A.1 provides a detailed, general taxonomy of graph-aware LLMs. We discuss related work on the broad landscape of attacks against Graph Neural Networks (GNNs) in Appendix A.2. Appendix A.3 gives an overview of the adversarial attacks NETTACK and METAATTACK, while Appendix A.4 details the algorithm for the node sequence injection attack on LLAGA. We present the results of our ablation studies in Appendix A.5, including the effect of varying the amount of perturbed features and the choice of base LLM on attack performance. Finally, Appendix A.7 shows examples of feature attacks and the template for the LLM corrector, and Appendix A.8 discusses the broader implications of our research.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data is a rapidly emerging and broad area of research, with a diverse range of approaches exploring how LLMs can be adapted or enhanced for graph-related tasks such as reasoning, representation, and inference. Despite their diversity, these approaches largely fall into two overarching categories defined by how they encode graphs for LLMs: *textual descriptions of graphs*, where graph information is encoded as natural language, and *learned graph projector*, where a projector encodes graph data into embeddings or structured forms that LLMs can directly process. In this section, we survey the methods spanning these encoding families for integrating graph data into LLMs. Figure 5 illustrates both families and Table 7 summarizes their key differences.

Graph Representation as a Textual Description In this first broad category, graph data is represented as natural language, allowing LLMs to process graph structures as textual descriptions. This approach has been explored extensively across many settings, with works such as (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a) converting graph data into textual formats, often paired with task-specific instructions to query LLMs. This family spans a wide range of instantiations. For example, Tan et al. (2023) introduced a unified framework for attributed graph embedding by generating random walk-based textual sequences from graphs, which are then used to fine-tune LLMs. Similarly, Zhao et al. (2023) focused on graph reasoning within text spaces, translating graph structures into natural language to facilitate LLM interaction. Extending this idea, Wei et al. (2024a) augmented textual representations of graphs with visual ones, enhancing LLMs’ reasoning over graph data. In multimodal settings, Liu et al. (2023) and Su et al. (2022) linked molecule graphs with textual descriptions, enabling advances in drug discovery and molecular retrieval. Furthermore, Huang et al. (2023a) investigated how incorporating graph structure into LLM prompts can enhance predictive performance, finding that LLMs tend to process graph-related prompts as contextual paragraphs

rather than explicitly recognizing graph structures. This further underscores the general importance of prompt design, as seen in studies such as (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024), which extend LLMs to heterogeneous and large-scale graph tasks. Complementing these efforts, (He et al. 2024) explored using LLM-generated explanations as features to improve both the performance and interpretability of LLMs on graphs.

Still representing graphs as textual descriptions, a related line of work integrates pretraining strategies with GNNs to strengthen LLMs’ graph reasoning across tasks. For instance, Chen et al. (2023a) proposed two LLM–GNN integration approaches: (1) using LLMs to enhance node features for node classification, and (2) employing LLMs as direct predictors for graph tasks. In a similar spirit, Chandra et al. (2020) used LLMs as text encoders and GNNs as graph structure encoders for tasks such as fake news detection in online communities. Likewise, Chen et al. (2023b) explored performing node classification without explicit labels by having LLMs annotate nodes, which then supervise a GNN, and Mavroumatis et al. (2023) used an LLM as a feature extractor for GNNs.

For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that combines LLMs and GNNs through a tunable side structure alongside each LLM layer, improving both training efficiency and inference speed. Similarly, Duan et al. (2023) introduced SimTeG, a simple yet effective approach that enhances textual graph learning with LLMs: the LLM is first fine-tuned on a downstream task such as node classification, after which node embeddings are obtained from the last hidden states of the fine-tuned model. In addition, Zhang (2023) proposed Graph-toolformer, which uses prompt engineering, powered by ChatGPT, to augment LLMs’ graph reasoning capabilities.

While these approaches make LLMs broadly accessible for graph-related tasks, they often struggle to fully capture the structural intricacies inherent in graph data. Moreover, their performance remains heavily dependent on effective prompt engineering, which can be a challenging process across tasks, requiring careful design and experimentation to obtain optimal results.

Graph Representation as a Learned Projector To address the challenges posed by textual representation approaches, the learned projector family offers a broad alternative, transforming graph data into embeddings or graph vectors that LLMs can process more directly. For instance, Chen et al. (2024) introduced LLAGA, a large language and graph assistant that retains the general-purpose nature of LLMs while transforming graph structures into graph tokens suitable for LLM processing. Similarly, (Liu et al. 2024; Qin et al. 2023) incorporated graph structure information into a pre-trained LLM through tailored disentangled graph neural network layers. Tian et al. (2024) proposed using graph neural prompts to encode knowledge graphs, transforming them into graph embeddings that LLMs can leverage during inference. Furthermore, Huang et al. (2023b) introduced a prompt-based node feature extractor (G-Prompt) for few-shot learning on text-attributed graphs, combining a graph adapter

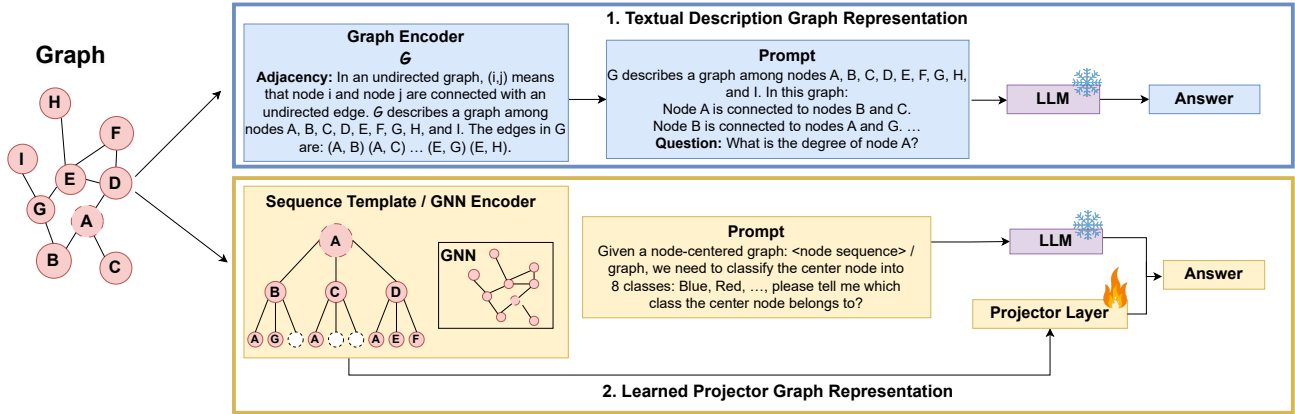


Figure 5: **Graph encoding-based adaptations of LLMs.** In the textual description graph representation, graphs are explained textually (top part). In the learned projector representation, graphs are encoded either by node templates that turns the graph into sequences or by GNNs (bottom part). Note that in both cases, the LLM is frozen. Only the projector that maps the graph into tokens is trained.

Table 7: Comparison of the two broad families of graph encoding-based adaptations of LLMs.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

(a learnable GNN layer) with task-specific prompts to capture neighborhood information and extract informative node features.

A further promising direction condenses graph information into fixed-length prefixes using graph transformers, which are attached to each LLM layer to enhance its reasoning capabilities (Chai et al. 2023). Similarly, Yang et al. (2021) developed GraphFormers, which seamlessly integrates text encoding and graph aggregation in an iterative workflow by nesting GNNs alongside the transformer blocks of a language model. In addition, Peng et al. (2024) developed ChatGraph, an interactive interface that lets users interact with their graphs through natural language, representing graphs as sequential paths and fine-tuning the LLM on the graph data to support graph analysis.

In this paper, we do not consider projector adaptations that require retraining the LLM, as they significantly increase computational costs without yielding substantial performance gains. Instead, we focus on the broadly adopted class of adaptation methods that build on **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). Beyond reducing computational cost, this choice offers greater flexibility for integrating with a wide range of LLM backbones, and it lets us exploit LLMs’ extensive knowledge and capabilities without the overhead of retraining while still adapting them to specific graph tasks. Throughout this work, we employ two representative projector methods that together span the dominant structural encodings in this family: **LLAGA** (Chen et al. 2024), which uses node-level templates with a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which employs a GNN-based projector to encode the graph.

This work. In this work, we do not separately study textual description methods, as they often struggle to fully capture the structural intricacies inherent in graph data. Such methods are generally confined to simpler tasks, like describing basic graph properties such as node degree, and become infeasible for larger graphs due to scalability limits: the token budget of prompts restricts the detailed descriptions required for larger structures, quickly exceeding allowable token counts. From a vulnerability standpoint, moreover, textual description methods introduce no unique attack surface. Because the graph is rendered as plain text (e.g., “Node A is connected to Node B”), these methods are exposed to conventional text-based adversarial attacks on LLMs rather than to the structural or feature perturbations specific to graph-based models. They therefore lack the complexity that would expose them to the novel, graph-specific vulnerabilities central to our investigation, and conclusions about their robustness follow largely from the existing literature on text-based attacks.

A.2 Attacks on GNN

GNNs are subject to a broad spectrum of attacks, including membership inference attacks (Olatunji, Nejd, and Khosla 2021; He et al. 2021; Conti et al. 2022), which aim to determine whether a specific node was part of the model’s training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which exploit the GNN’s outputs and learned embeddings to in-

fer sensitive or missing node attributes; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which seek to extract global properties of the entire graph; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which degrade performance by introducing subtle perturbations to the graph or node features; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which aim to reconstruct the underlying graph structure from learned embeddings or model outputs.

Among these, adversarial attacks are of particular concern because of their direct impact on model performance across settings. Such attacks introduce small, carefully crafted *perturbations* into the graph—modifying node features or altering edges—with the aim of significantly degrading the GNN’s performance. They fall broadly into two types: **poisoning attacks**, which act during the *training phase* by manipulating the training data, and **evasion attacks**, which act at *test time*, where the adversary perturbs the input graph to mislead the model’s predictions. For instance, Zügner, Akbarnejad, and Günnemann (2018) demonstrated that adversarial perturbations on graph structures or node features can significantly degrade the performance of GNNs. Their method, **NETTACK**, used a greedy optimization technique to compute the minimal perturbations that maximize the model’s training-time loss, effectively identifying the smallest feature or structural changes needed to disrupt predictions. Li et al. (2023) revisited adversarial attacks on graph data from a data-distribution perspective, showing how adversarial examples can significantly shift the underlying data distribution, and proposed a novel defense to mitigate such effects. In a similar vein, Wu et al. (2019) offered a comprehensive analysis of adversarial examples on graph data, with deep insights into both attacks and defenses; focusing on the vulnerabilities of GNNs to small perturbations, they proposed defenses designed to detect and counteract adversarial modifications and thereby improve robustness. **METAATTACK** (Zügner and Günnemann 2019) performs both poisoning and evasion attacks on GNNs by treating the graph structure as an optimizable hyperparameter. The attack modifies the structure directly via gradient descent, formulated as a bi-level optimization task, and relies on meta-gradients that capture how small structural perturbations affect the attacker’s post-training loss, enabling precise updates that degrade model performance.

In this paper, we adopt **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial attack methods that together span both poisoning and evasion strategies, so that our investigation covers the principal regimes of graph adversarial attacks. Poisoning attacks target model integrity during training, whereas evasion attacks compromise performance at inference or test time.

A.3 How NETTACK and METAATTACK Work

NETTACK perturbs the graph structure by adding and removing edges between nodes while preserving the degree distribution of the perturbed graph. It uses a greedy optimization technique to compute the minimal perturbations that maximize the model’s training-time loss with respect to a

surrogate model—a linearized 2-layer graph convolutional network (GCN) that approximates how structural changes affect classification. The primary objective of NETTACK is to increase the model’s confidence in incorrect classifications by maximizing the distance between the logits of the perturbed and clean graphs. To this end, NETTACK identifies candidate edges around a target node and evaluates a score function for each connected edge, indicating how strongly altering that edge will affect the target node’s classification. The highest-scoring edge is selected, and the graph is updated by adding or removing it. Since NETTACK is a targeted attack (misclassifying a single specified node), we follow (Zügner and Günnemann 2019) to convert it into an untargeted attack (degrading the model’s overall node classification performance): we randomly select a test node, attack it with NETTACK while considering all nodes in the network, and aggregate the results over the test nodes.

METAATTACK optimizes the graph structure directly via gradient descent, treating the adjacency matrix (the graph structure) as a hyperparameter. It frames the attack as a bi-level optimization problem in which the inner level minimizes the training loss on the original graph and the outer level minimizes the loss on the poisoned graph. Like NETTACK, it relies on a surrogate model—a linearized two-layer GCN—to approximate how structural changes affect performance. A score function evaluates the impact of each perturbation, and the attack flips the sign of meta-gradients (gradients with respect to the hyperparameters) for connected nodes to indicate which edges to remove. The algorithm then greedily selects the highest-scoring perturbations, adding or removing edges, to maximize the attack’s impact.

For both methods, we apply the same unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018), which ensures that the graph’s degree distribution changes only slightly; accordingly, we perturb only 10% of the edges.

In our poisoning experiments with LLAGA, we use the perturbed graph structure obtained from NETTACK or METAATTACK to generate the node-level sequences and retrain the projector that converts those sequences into token embeddings. Analogously, for GRAPHPROMPTER, we use the perturbed structure, re-encode the graph with a graph attention network (GAT), and retrain the projector on the perturbed graph. For evasion attacks, the graph-aware LLM is trained on unperturbed data, with perturbations introduced only at inference. These procedures depend only on each model’s standard encoding pipeline and thus carry over readily to other frozen-LLM projector models.

A.4 Algorithm for Node Sequence Template Injection Attacks

We present the algorithm for the node sequence template injection attack in LLAGA in Algorithm 1; the same principle applies to any encoding that constructs fixed-shape neighborhood templates. The model builds a *fixed-shape* computational tree for each node u , sampling k neighbors per node up to depth d (lines 1–14). When a node has fewer than k neighbors, placeholders are inserted, creating an attack surface. We exploit this by injecting malicious nodes into placeholder

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k

Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$ 
       neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow$  2-hop neighborhood of  $u$ 
21:     if AttackStrategy = NI then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow$  Next node from  $V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if AttackStrategy = SI then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:         break ▷ Only use first placeholder
34:       end for
35:     else if AttackStrategy = MSI then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow$  Get  $i$ -th node from  $S$ 
40:          $p \leftarrow$  Get  $i$ -th placeholder from  $P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

positions through graph manipulation. We propose three complementary strategies: *Non-Adjacent Injection (NI)* inserts nodes outside the 2-hop neighborhood of u (lines 21–27);

Supernode Injection (SI) injects a high-degree node into one placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all placeholders (lines 35–43). In each case, the uninformative injected nodes corrupt the node sequence and, in turn, the target’s representation, impairing training of the projection layer.

A.5 Ablation Studies

Here, we present the results for the ablation studies presented in the main paper.

Ablation Results: Varying Feature Perturbation In Figure 6, we report the performance of the two graph-aware LLMs, LLAGA and GRAPH PROMPTER, as the fraction of perturbed features is varied. Increasing the proportion of perturbed features consistently degrades accuracy for both models, and the fact that this holds across both encoding strategies points to a general sensitivity to feature-level attacks throughout the paradigm.

Ablation Results: Does the Choice of LLM Affect Attack Performance? The results in Table 8 show that the choice of base LLM has minimal impact on both clean performance and adversarial vulnerability within the LLAGA framework. Across Cora, Citeseer, and PubMed, models built on Vicuna-7B and Llama2-7B achieve nearly identical clean accuracy and exhibit similar degradation under both poisoning and evasion attacks. For instance, under METAATTACK evasion, Cora performance drops to 0.44 ± 0.06 for Vicuna-7B and 0.43 ± 0.08 for Llama2-7B—differences well within one standard deviation. The same pattern holds across all datasets and attack types, indicating that attack effectiveness is largely independent of the base LLM and that our conclusions are robust to the choice of backbone.

A.6 Results on the ArXiv Dataset

To assess how our attacks generalize to large, real-world graphs, we evaluate on the large-scale ArXiv dataset, which comprises 169,343 nodes, 1,166,243 edges, and a sparsity of 0.81. This experiment probes the effectiveness of node sequence template attacks at a scale well beyond the smaller benchmarks. As shown in Table 9, we observe a pronounced gap between the success of poisoning and evasion attacks, with poisoning consistently outperforming evasion across all three injection strategies—a reversal of the pattern seen on Cora, Citeseer, and PubMed. This suggests that ArXiv possesses inherent structural characteristics that render it more vulnerable to poisoning yet more robust to evasion, illustrating how attack behavior can vary with graph scale and topology. Despite ArXiv’s very low sparsity, inserting even a single supernode can substantially affect model performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The examples in the prompt, shown in Appendix A.7, illustrate our adversarial attacks on real features from the Cora dataset, though the same mechanism applies to any textual node attributes. The “input” is

the text the LLM receives once the attack has been applied. Importantly, while a human reader perceives the original and perturbed text as visually identical or nearly so, the LLM processes a different sequence of Unicode characters or a different character order, which can alter its behavior. We illustrate this in Table 10 for the homoglyph attack and Table 11 for the reordering attack.

Feature Corrector Defense Template We designed a feature corrector module to mitigate imperceptible attacks, including homoglyph substitutions and character reorderings, as well as related textual perturbations. The module is implemented as a prompt-based defense using LLMs, making it broadly deployable without model retraining. The prompt is carefully crafted to instruct the LLM to act as a linguistic validator focused on detecting and correcting subtle, human-imperceptible perturbations.

The template for our prompt is shown below. It states the task explicitly and provides a clear output format, which substantially improves the LLM’s performance and generalizes across inputs.

A.8 Detailed Discussion

Our findings surface several insights into the vulnerabilities and design considerations of graph-aware LLMs that we expect to hold broadly across the paradigm. As the integration of graph-structured data into LLMs continues to gain traction across graph tasks, it is essential to examine not only the performance benefits but also the vulnerabilities of these emerging architectures. Our systematic investigation of adversarial attacks on two representative graph-aware LLMs—LLAGA and GRAPH PROMPTER—reveals significant weaknesses, especially under poisoning and evasion attacks traditionally used to undermine GNNs. Because these two models span the dominant encoding strategies, the observations below are likely to generalize to related models. Our key observations are discussed next.

Impact of Graph Encoding on Adversarial Vulnerability

A key observation from our results is that the way graph structure is integrated into an LLM can inadvertently increase susceptibility to adversarial attacks—a design-level effect that is not specific to any one model. Specifically, the node sequence template employed in LLAGA creates a vulnerability that adversaries can exploit to substantially degrade performance. While this particular attack surface is realized in LLAGA, it reflects a broader tradeoff in graph-aware LLM design: node-level templates offer an effective means of incorporating graph structure and achieve strong performance on graph tasks, yet they also expose the model to new attack vectors absent in traditional GNNs. In contrast, GRAPH PROMPTER, which uses a GNN-based projector for graph encoding, exhibits comparatively greater resilience, underscoring the potential of hybrid architectures that combine the strengths of both LLMs and GNNs.

Sensitivity of Graph-aware LLMs to Feature Perturbations

Beyond vulnerabilities in the graph encoding techniques, our results highlight the effectiveness of feature

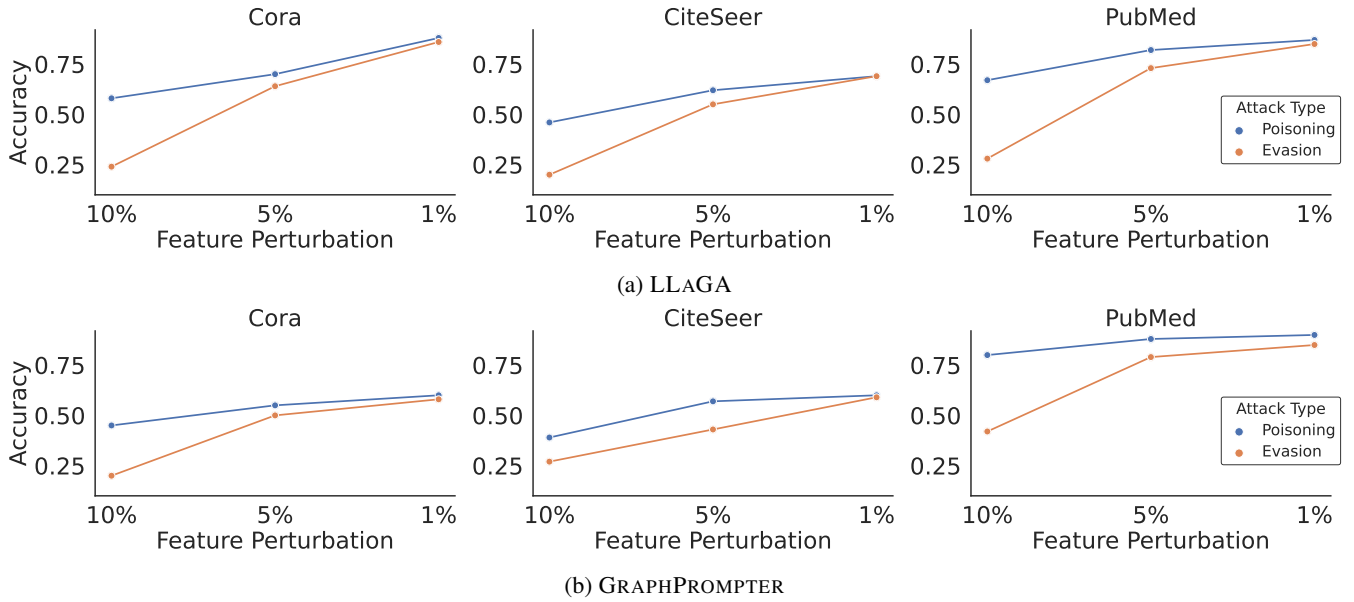


Figure 6: Performance of the graph-aware LLMs as the percentage of perturbed features is varied, shown across models and datasets. Lower accuracy indicates a more successful attack.

Table 8: Impact of changing the base model on clean performance and on robustness under different attacks, shown for LLaGA across all datasets. The results demonstrate that the choice of base model does not significantly affect susceptibility to attacks.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

perturbation attacks across both graph-aware LLMs. Despite their differing structural encodings, both LLaGA and GRAPHPROMPTER exhibit pronounced sensitivity to imperceptible perturbations of node features, pointing to a paradigm-wide trend: while these models excel at handling textual data, their robustness to adversarial perturbations—especially those targeting node features—remains a critical, broadly shared challenge. The ability of adversaries to degrade performance by subtly and undetectably altering node attributes calls into question the integrity of graph-aware LLMs across the many security-sensitive applications where they may be deployed. While text sanitization techniques, such as those discussed by (Boucher et al. 2022), offer partial defenses, countering imperceptible attacks like homoglyph-based manipulations remains difficult in general. As we propose, using an LLM as a corrector—prompting the model to detect and repair suspicious or malformed inputs before downstream processing—offers one broadly applicable way to mitigate such vulnerabilities.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Our findings reveal that the efficacy of poisoning and evasion attacks against graph-aware LLMs is not uniform across datasets. While attacks are particularly effective on Cora, Citeseer, and PubMed, we observe a more nuanced interplay between attack strategy and graph structure on the ArXiv dataset. This variation underscores the importance of dataset characteristics—scale, density, and topology—in assessing the vulnerabilities of graph-aware LLMs. As these models are increasingly deployed across diverse real-world settings, their vulnerabilities should be evaluated not only in terms of algorithmic design but also with respect to the specific domains and data to which they are applied.

Implications for Graph-aware LLM Development The vulnerabilities exposed in this study carry broad implications for the future development and deployment of graph-aware LLMs. Given their growing adoption for tasks once reserved for GNNs, it is crucial to weigh the tradeoffs between perfor-

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgprir heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly allcoct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llection plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: The prompt template used for the LLM-based feature corrector module. It explicitly defines the role, task, and expected output format for the LLM.

Table 9: Results of the node sequence template injection attack across datasets and injection strategies, including the large-scale ArXiv graph. *NI*, *SI*, and *MSI* denote the non-adjacent, single supernode, and multiple supernode injection attacks, respectively. Clean refers to model performance without any attack.

Data	Attack	NI	SI	MSI
Cora	Clean	————	0.89 ± 0.07	————
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	————	0.71 ± 0.04	————
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	————	0.90 ± 0.04	————
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	————	0.73 ± 0.02	————
	Poisoning	0.50 ± 0.03	0.43 ± 0.04	0.42 ± 0.03
	Evasion	0.71 ± 0.06	0.69 ± 0.02	0.60 ± 0.03

mance and vulnerability when designing these models. As the paradigm continues to evolve, there is a pressing need for more robust encoding techniques and defenses against adversarial manipulation, particularly for feature perturbations. Moreover, understanding how adversarial attacks transfer from GNNs to graph-aware LLMs opens broad avenues of research into protecting these models against increasingly sophisticated adversaries across settings.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** Adversarial attacks on GNNs have focused predominantly on structural perturbations, since feature-based perturbations typically fail to change the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019) and are therefore ineffective. Consequently, there are *no known defenses against feature-based attacks on GNNs*, owing to their limited effectiveness. As demonstrated in the main paper, however, feature perturbation attacks prove broadly effective in the graph-aware LLM paradigm. This introduces a new challenge: feature-based attacks that were once ineffective against GNNs can now successfully manipulate model behavior across graph-aware LLMs. The shift highlights a general need for defenses that address both structural and feature-based perturbations. While text sanitization techniques—such as using optical character recognition (OCR) as a preprocessing step before encoding text for the LLM—offer partial defenses, countering imperceptible attacks like homoglyph-based manipulations remains difficult, as OCR is imperfect and misinterprets homoglyphs at a higher rate than unperturbed text (Boucher et al. 2022). This motivated our design of an LLM-based self-correcting mechanism for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based adversarial attacks transfer from GNNs to graph-aware LLMs, particularly under evasion, one might reasonably expect defenses developed for GNNs to carry over to this paradigm. This assumption, however, does not hold in general. Unlike traditional GNN settings, *graph-aware LLMs are additionally susceptible to feature perturbations, which renders these defenses ineffective.*

One such defense mechanism in GNNs makes the adjacency matrix trainable, allowing edge weights to adapt during training to mitigate adversarial perturbations. This approach assigns lower weights to edges connecting dissimilar nodes and higher weights to edges linking nodes with greater feature similarity. A broad class of **preprocessing-based** defenses builds on this idea, for example by removing edges between nodes with low feature similarity (Wu et al. 2019) or introducing randomized edge dropping (Dai et al. 2018). All of these defenses, however, rely heavily on clean, unperturbed node features to compute reliable similarity scores. When node features are themselves perturbed, as in our attacks, such defenses become ineffective and may even worsen the problem by introducing incorrect edges. We illustrate this effect on a random node in the Cora dataset (Figure 8). While the clean feature similarity graph preserves some original edges (alongside a few spurious connections), the edges reconstructed from perturbed features preserve none of the original connections, yielding entirely incorrect reconstructions.

Another line of work focuses on decoupling reliance on the graph structure. For instance, (Shanthamallu, Thiagarajan, and Spanias 2021) proposed a defense that trains a surrogate predictor depending solely on node features and uses an uncertainty-matching strategy to extract graph information from the GNN. Similarly, Wu et al. (2021) introduced a co-training framework that trains submodels independently on feature and structural views, enabling knowledge distillation by adding confident unlabeled data to the training set. Yet these approaches, too, assume access to unperturbed node features—an assumption that fails in our setting, where both structure and features are adversarially perturbed. Consequently, structural defense strategies developed for GNNs are not directly applicable to the graph-aware LLM paradigm, which calls for new defenses tailored to it.

Table 10: An illustration of the imperceptible homoglyph attack, representative of the broader class of character-substitution perturbations. Although the LLM processes the perturbed text as distinct characters, the visual output is nearly identical to a human, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgprir huristic fr discvring prtln squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrns in grups f prtln squncs r in us tht r bsd on fitting th prmtrs f sttistical mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: An illustration of the imperceptible reordering attack, representative of the broader class of character-order perturbations. Although the LLM processes the perturbed text as distinct characters, the visual output is nearly identical to a human, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplclpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

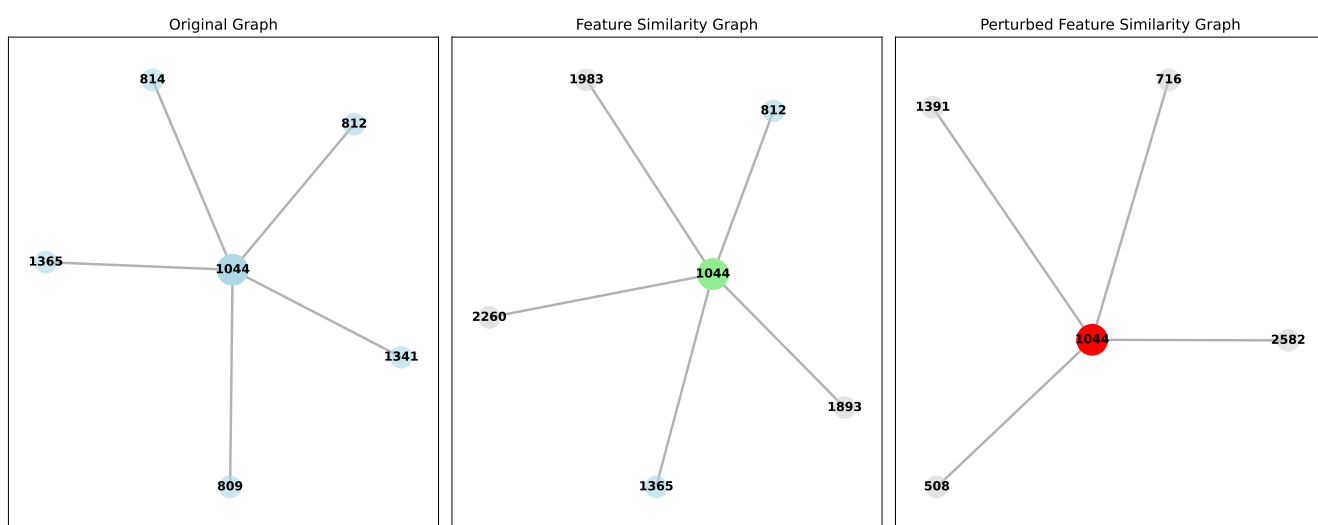


Figure 8: Edges reconstructed for a representative random node (1044). **Original (left)** shows the ground-truth edges from the original graph. **Feature Similarity (middle)** shows the edges reconstructed from clean node features, where blue nodes indicate connections preserved from the original graph. **Perturbed Feature Similarity (right)** shows the edges reconstructed from perturbed node features.